# COMP5349– Cloud Computing Week 9: CodePipeline & Decoupled Architecture

Dr. Ying Zhou

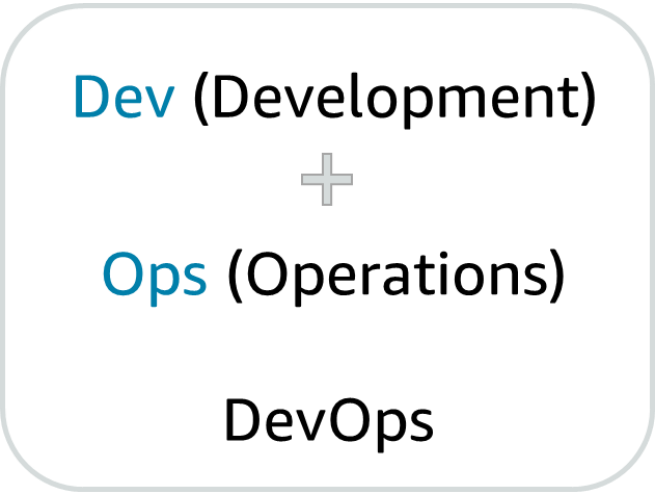
The University of Sydney

# Table of Contents

# DevOps and CI/CD

# DevOps

- DevOps is a set of practices, cultural philosophies, and tools that aim to integrate and automate the processes between software development (Dev) and IT operations (Ops).
- The ultimate goal is to enable faster and more reliable software delivery with improved collaboration and communication between teams.
- It focuses on automation, collaboration, fast feedback, and shared responsibility across the software delivery lifecycle.



# The problem DevOps tries to solve

- Traditional software delivery challenges
  - Developers write code, operations teams deploy and run it
  - Manual deployment steps are slow and error-prone
  - Bugs may be found late, after many changes have accumulated
  - Environment differences can cause “it works on my machine” problem
  - Infrastructure maybe changed manually, making systems hard to reproduce

# DevOps Culture

- Increased transparency, communication and collaboration between teams
- Shared responsibilities
- Autonomous teams
- Fast feedback
- Automation

https://www.atlassian.com/devops/what-is-devops/devops-culture

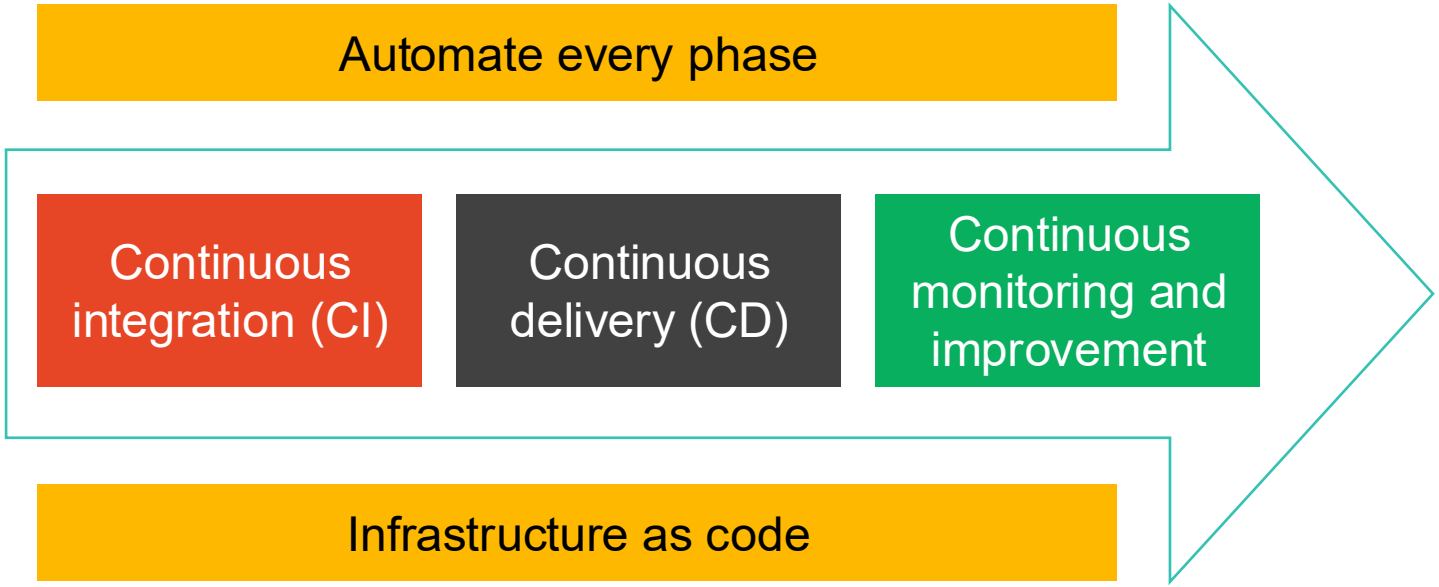
# DevOps Best Practice

- Agile Project Management
- Shift left with **CI/CD**
- Build with the **right tools**
  - DevOps life cycle: discover, plan, build, test, monitor, operate, continuous feedback
- Implement automation
- Monitor the DevOps pipeline and applications
- Observability
  - three pillars of observability are **logs**, **traces**, and **metrics**.
- Change the culture
  - You build it you run it

https://www.atlassian.com/devops/what-is-devops/devops-best-practices

# AWS Supported DevOps practices

- Microservice architecture
- Continuous integration and continuous delivery (CI/CD)
- Continuous monitoring and improvement
- Automation focused
- Infrastructure as code



# DevOps tools

**CI/CD**

- AWS CodePipeline
- AWS CodeBuild
- AWS CodeDeploy
- AWS CodeConnections

**Microservices**

- Amazon Elastic Container Service (Amazon ECS)
- AWS Lambda
- AWS Fargate

**Platform as a Service**

- AWS Elastic Beanstalk

**Infrastructure as Code**

- AWS CloudFormation
- AWS OpsWorks
- AWS Systems Manager

**Monitoring and logging**

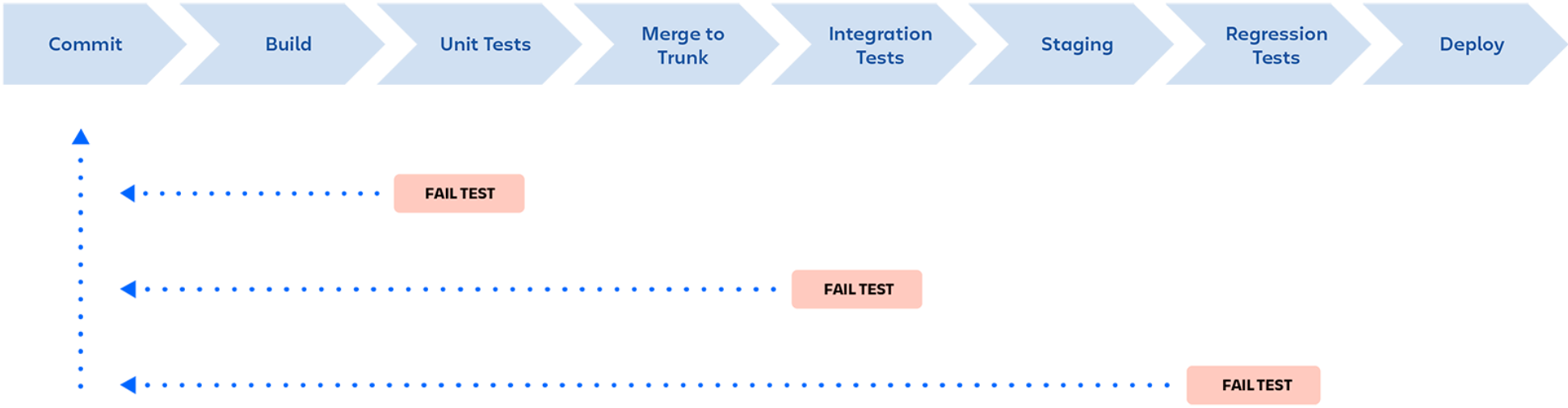
- Amazon CloudWatch
- AWS CloudTrail
- AWS X-Ray
- AWS Config

# Continuous Integration

- A software development practice where members of a team use a **version control system** and frequently integrate their work to the same location, such as a main branch. Each change is built and verified to detect integration errors as quickly as possible. Continuous integration is focused on automatically building and testing code
  - Version control system to manage the code change
  - A CI server to manage the automated process of building, testing, and validating those changes

# Continuous Delivery/Continuous Deployment

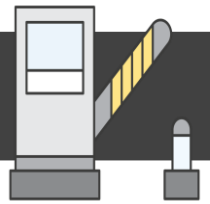
- Continuous Delivery extends CI by including automatically releasing software to a repository.
- Continuous Deployment extends the process further by adding the step of automatically deploying software to production.



# Understanding CI/CD


Continuous integration

Continuous delivery



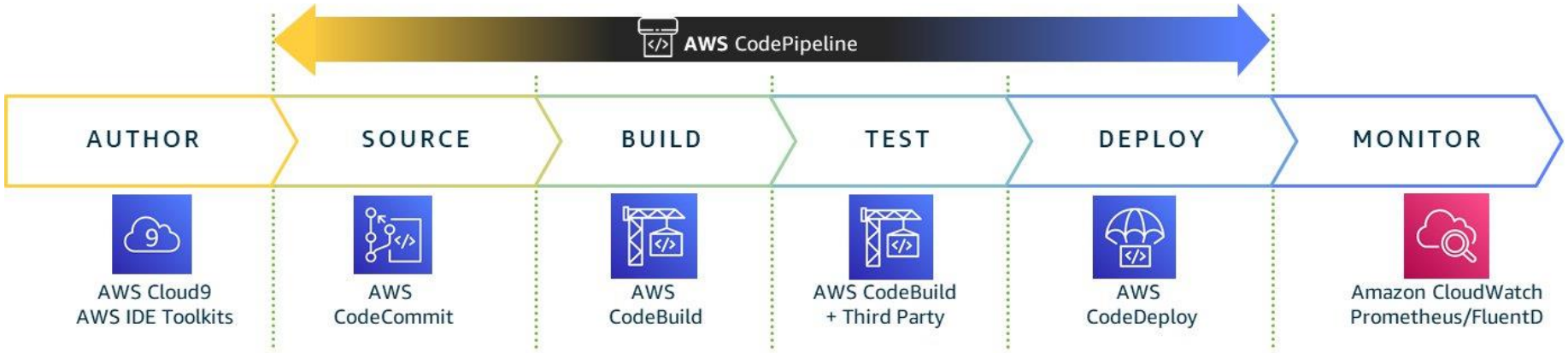
Manual
Intervention

Continuous deployment

# AWS CodePipeline

# CI/CD on AWS

- CI/CD is usually pictured as a pipeline with each stage structured as a logical unit in the delivery process
  - Specialized tool for each stage
  - Manual checking can be added at various points
  - Some stages maybe skipped for certain pipelines

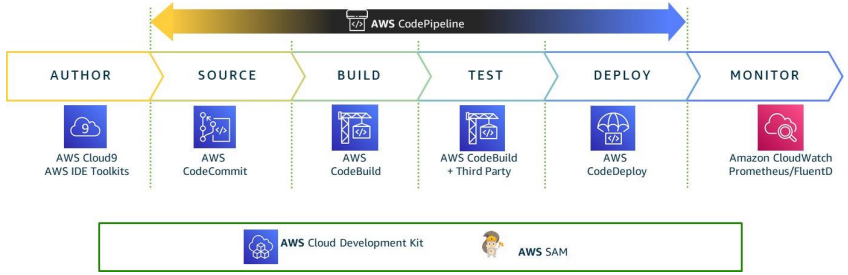



# AWS CI/CD pipeline components

- AWS provides several services that can be combined to build CI/CD pipelines
  - AWS CodeCommit/GitHub/S3**:** source repository or source artifact
  - AWS CodeBuild: build, test or validation commands
  - AWS CodePipeline: pipeline orchestration
  - AWS Code Deploy / CloudFormation: deployment
- The pipeline can be used to manage application code and infrastructure code
  - Pipelines for infrastructure code may not need to include all stages
  - The pipelines in the lab contains only two stages: source and deploy

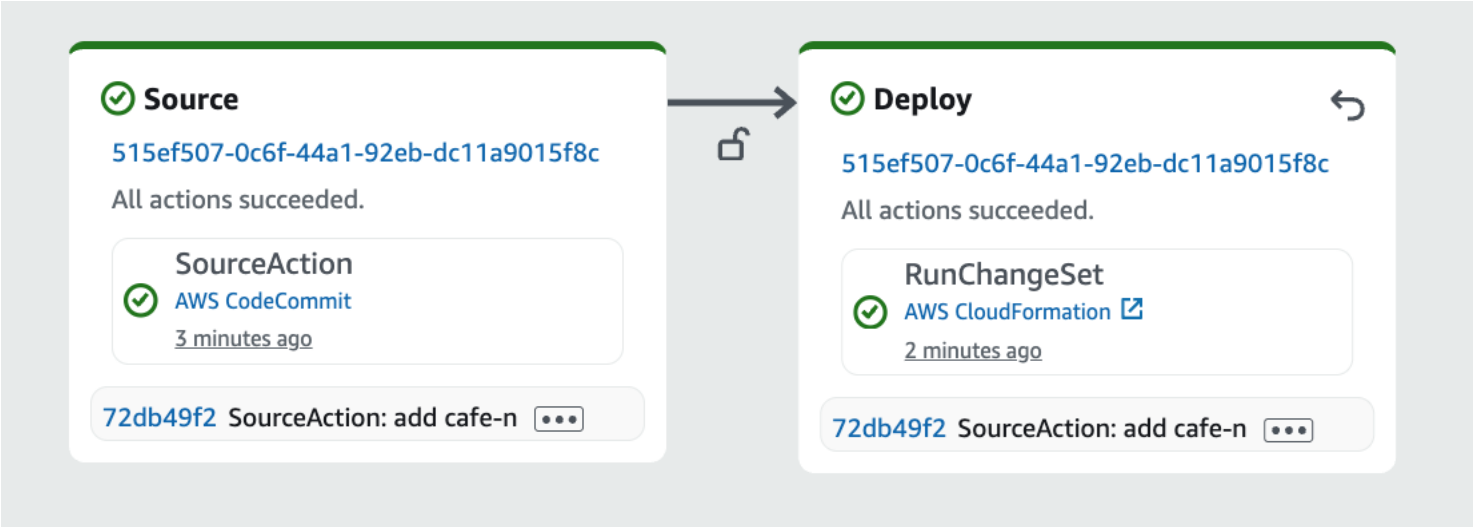
# CodePipeline

- A service that allows users to automate steps required to release/deploy software on AWS easily
- It coordinates stages such as source, build, test, approval, and deploy.
- CodePipeline does not usually do all work itself.
  - It invokes other services, such as CodeBuild, CloudFormation, CodeDeploy, ECS, or Lambda.
- In this week's lab, CodePipeline invokes CloudFormation to update infrastructure.



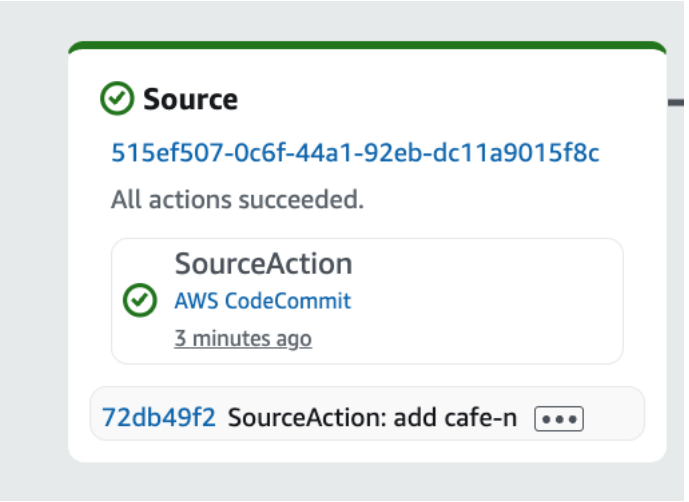
# Pipeline

- A *pipeline* is a workflow construct that describes how software changes go through a release process.
- Each pipeline is made up of several stages
  - The simplest pipeline may contain only two stages

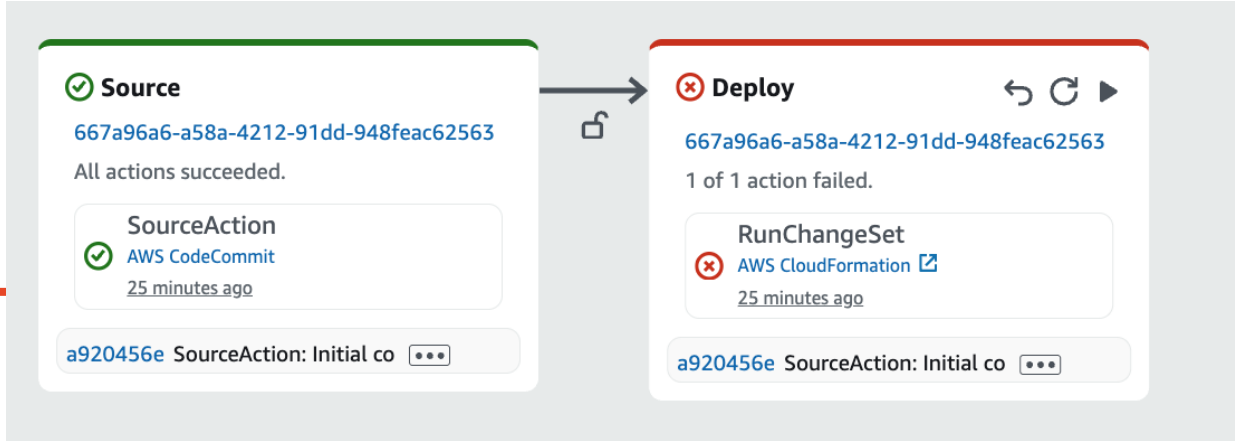


# Stages

- A stage is a major step in a pipeline, such as Source, Build, Test, or Deploy.
- Each stage contains one or more actions.
- An action performs a specific task on an artifact.
  - Source action: retrieves code or templates.
  - Build action: runs commands.
  - Deploy action: updates an application or CloudFormation stack.
  - If an action fails, the pipeline execution stops at that stage.



# Pipeline execution




- A pipeline execution is one run of the pipeline.
- It moves through the stages in order and complete actions defined in the stages, which may end up with a successful execution or failed execution
- A pipeline can start manually or be triggered by a source change, such as a commit.
- If an action fails, the execution stops and the failed stage must be investigated.
- For CloudFormation deployment failures, check:
  - The failed CodePipeline action
  - The CloudFormation stack events

| | | | | | |
|---|---|---|---|---|---|
| ○ | CafeAppPipeline | Failed | **SourceAction** – 72db49f2: add cafe-network template | 3 minutes ago | View details |
| ○ | CafeNetworkPipeline | Succeeded | **SourceAction** – 72db49f2: add cafe-network template | 3 minutes ago | View details |

# Debugging a failed infrastructure pipeline

- A failed CodePipeline execution may be caused by
  - Invalid CF template syntax
  - Invalid resource properties
  - Missing IAM permissions
  - Resource name conflict
  - Dependency or ordering problem
  - Etc
- Debugging usually requires checking more than one place:
  - Pipeline execution detail
  - CF Stack events
  - Cloud watch logs

# Validate CF templates before deployment

- Some CloudFormation errors are easier to fix before stack creation or update.
- Basic validation:

```
aws cloudformation validate-template \
  --template-body file://template.yaml
```

- Static analysis with cfn-lint:

```
cfn-lint template.yaml
```

- These checks can be run locally or in a CodeBuild stage

# Preview infrastructure changes

- A CloudFormation update can create, modify, or delete resources.
- A change set shows the proposed changes before they are applied.
- This is useful when:
  - updating production infrastructure
  - reviewing whether a change will replace a resource
  - adding manual approval before deployment
- A safer pipeline can use:

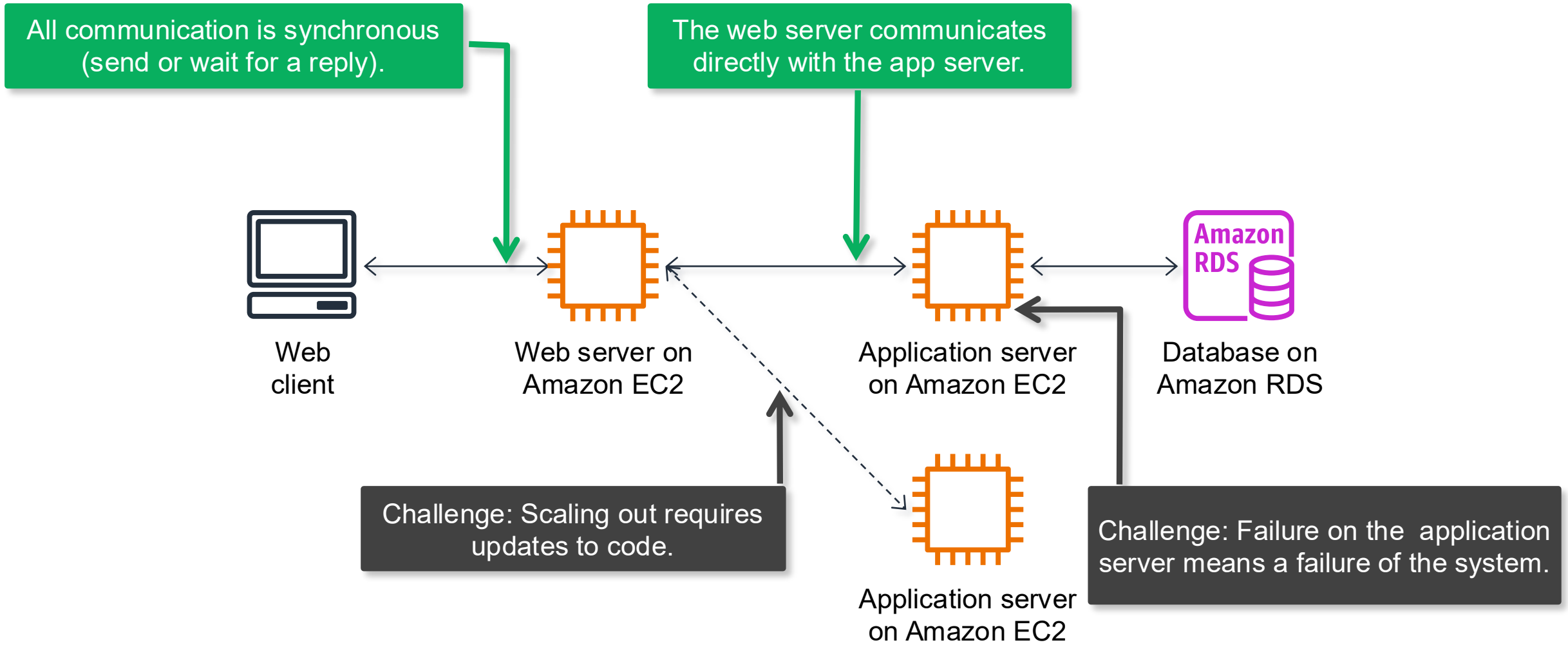
  create change set
  - review / manual approval
  - execute change set

# A safer infrastructure CodePipeline structure

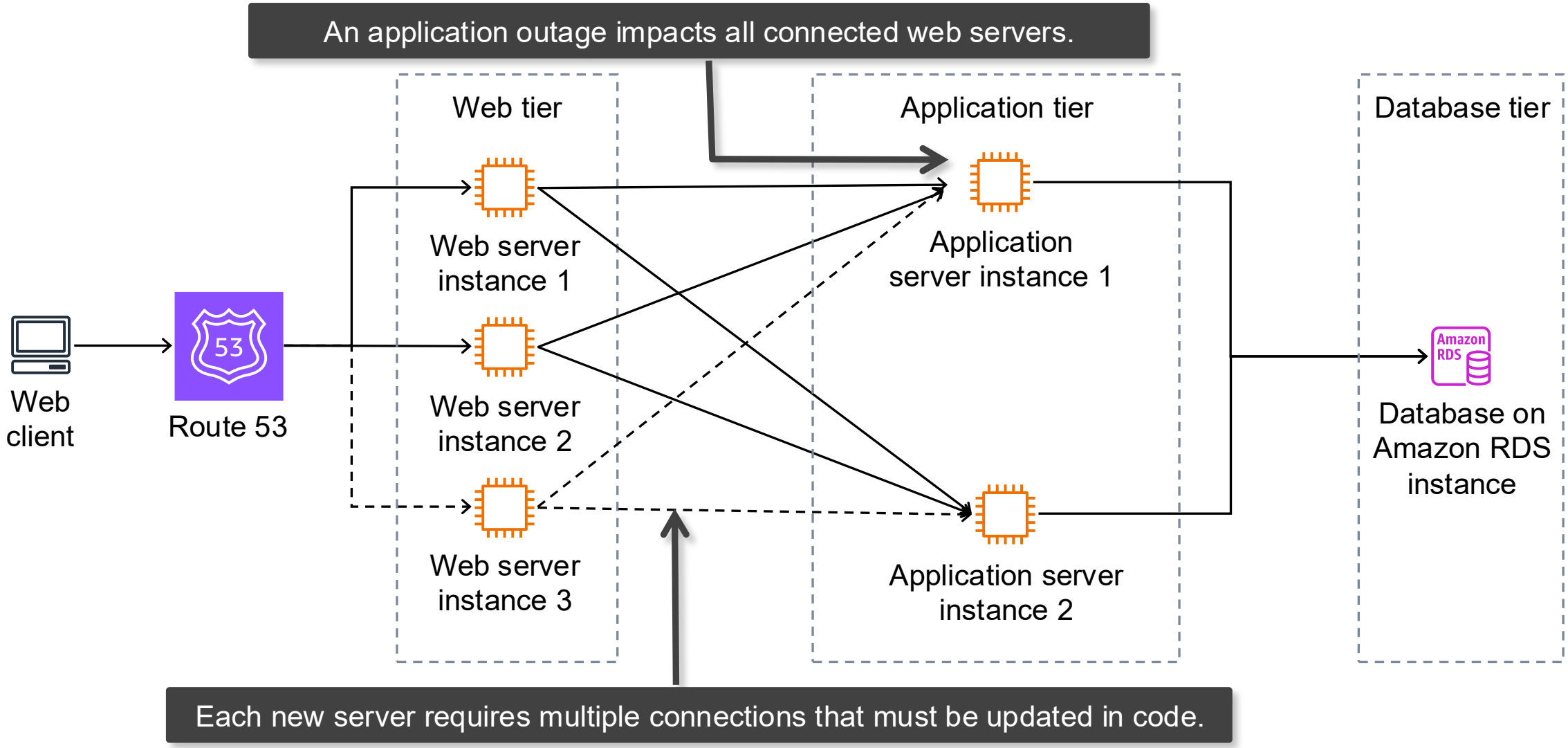
| Stage | Action | Purpose |
| --- | --- | --- |
| Source | CodeCommit / GitHub / S3 | Get the CloudFormation template |
| Build / Validate | CodeBuild | Run cfn-lint or other checks |
| Prepare Change Set | CloudFormation: Create or replace change set | Ask CloudFormation to calculate proposed changes |
| Approval | Manual approval | Human reviews the change set |
| Deploy | CloudFormation: Execute change set | Apply the approved infrastructure changes |

# Decoupling Software Architecture

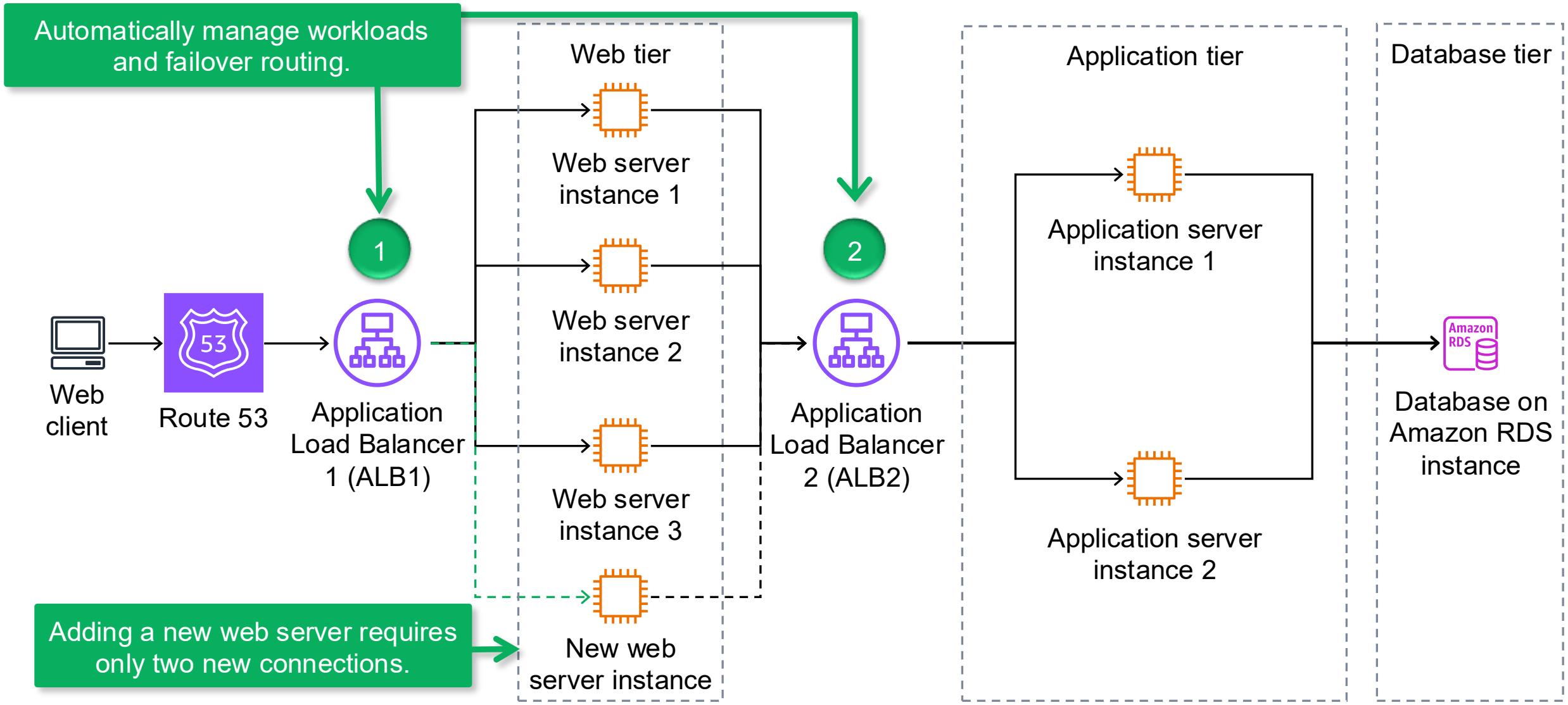
# Tight coupling in a three-tier architecture



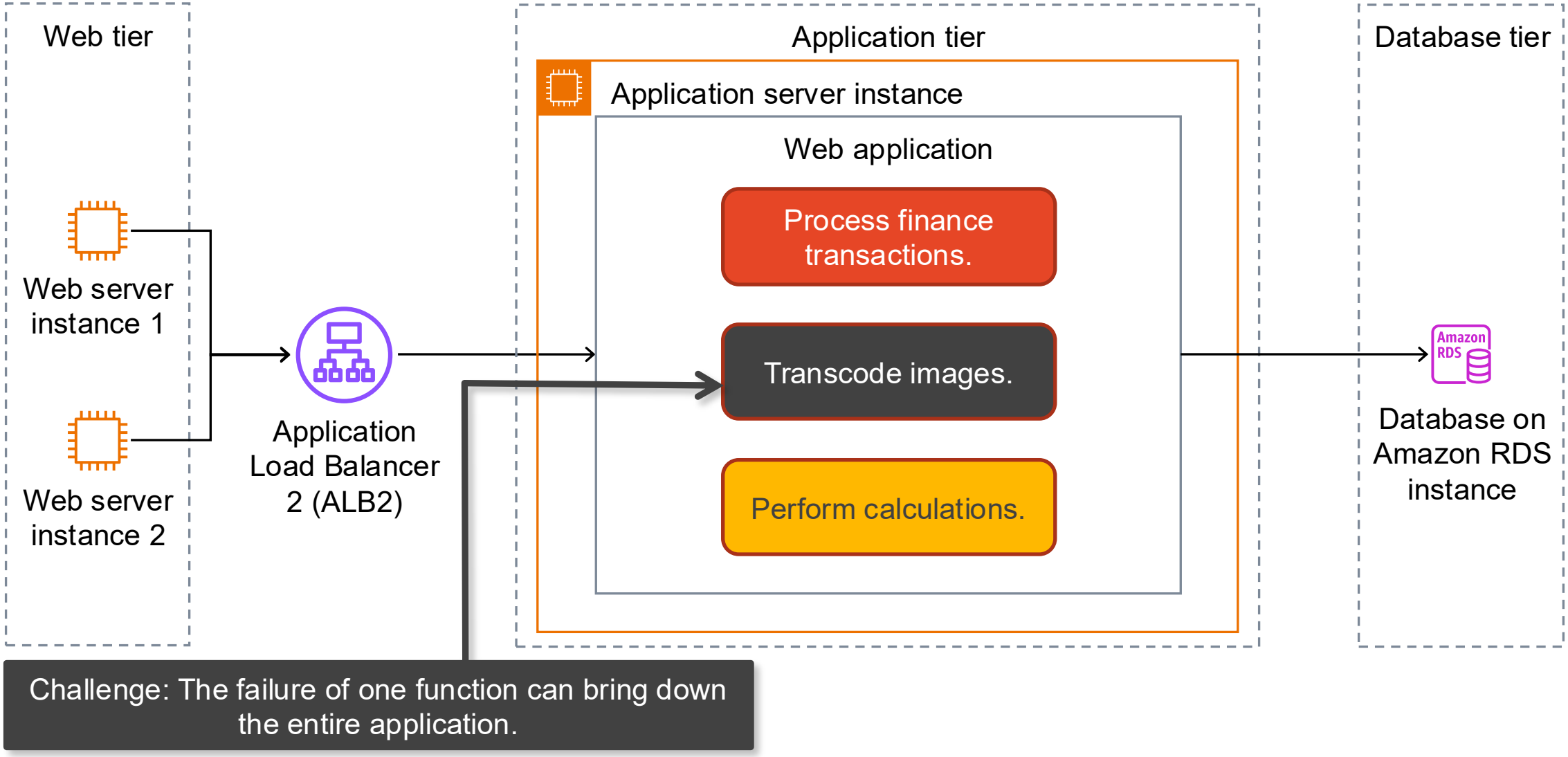
# Tight coupling increases the complexity of scaling



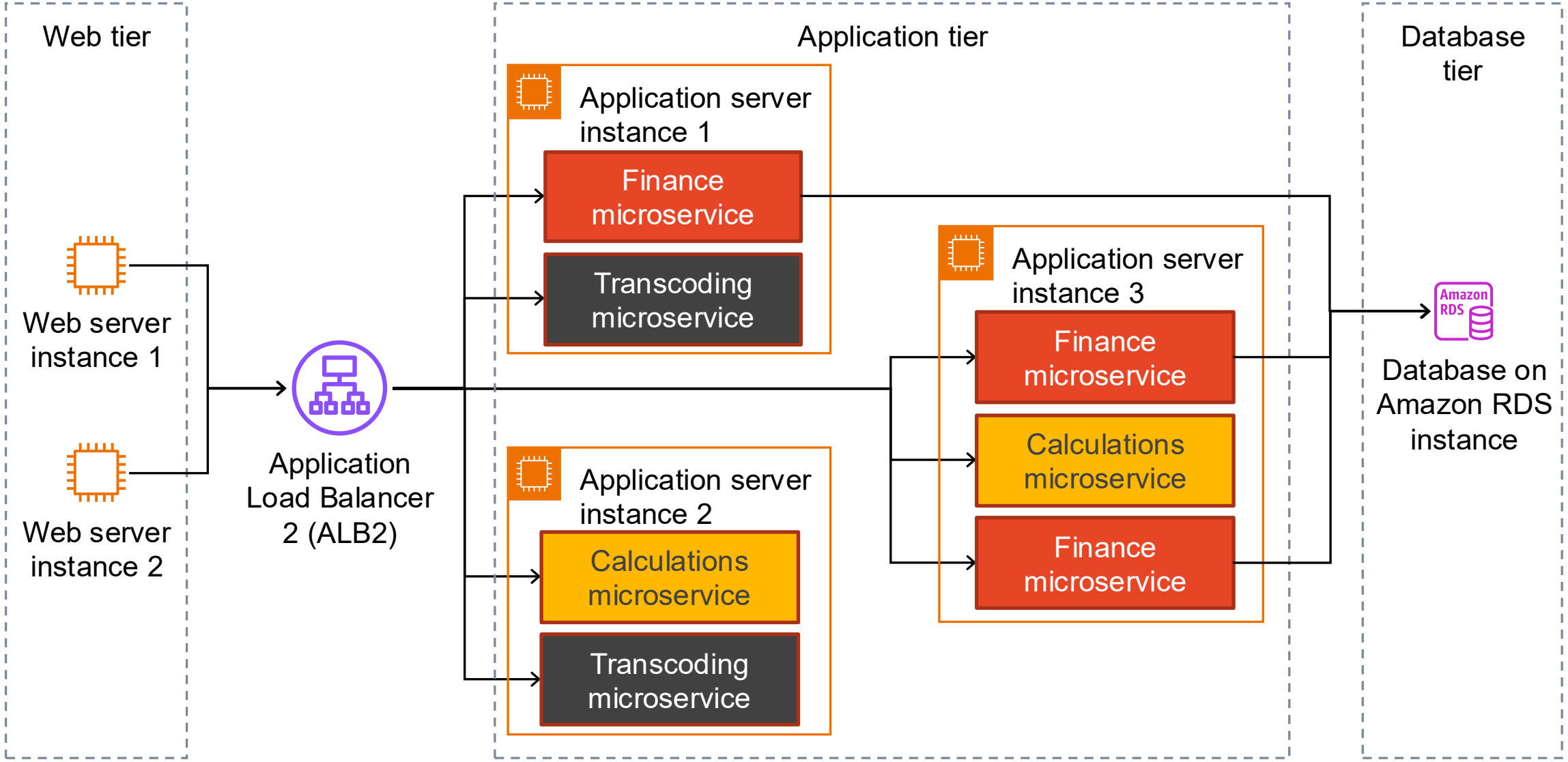
# Loose coupling between tiers



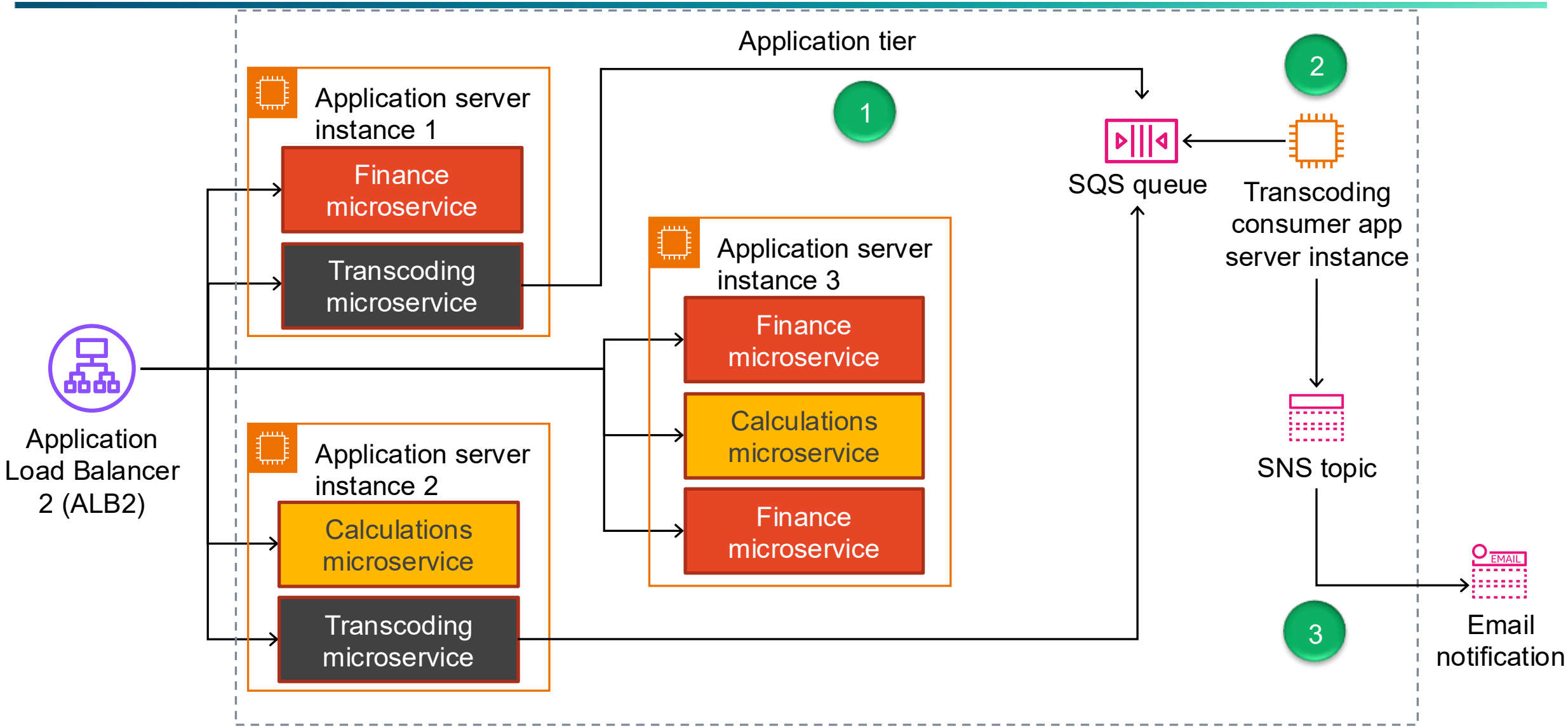
# Tight coupling within an application



# Loose coupling: Microservice architecture



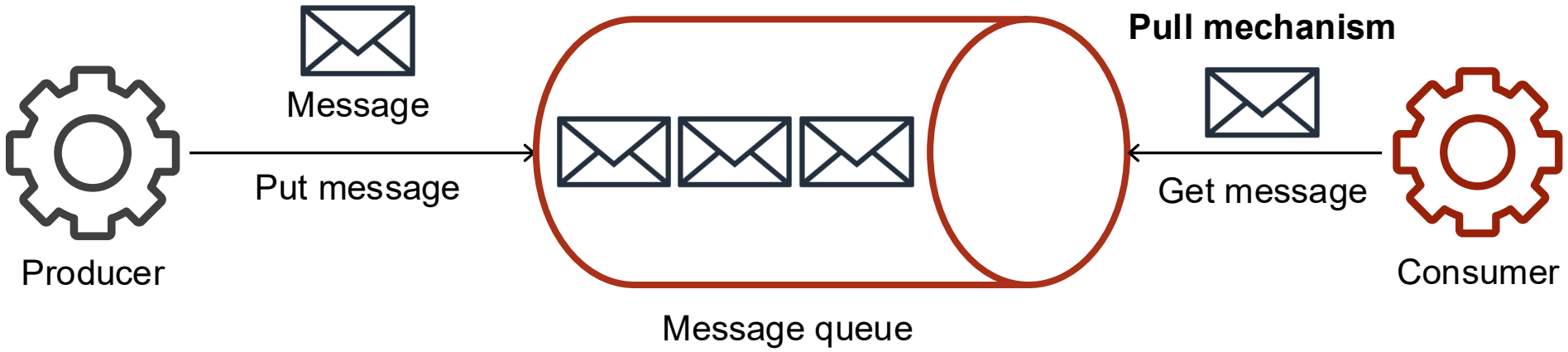
# Request offloading: Amazon SQS and Amazon SNS



# Amazon SQS

# Point-to-point messaging

- You can decouple applications asynchronously by using point-to-point messaging.
- Using point-to-point messaging when the sending application sends a message to only one specific receiving application.
- The sending application is called a *producer*.
- The receiving application is called a *consumer*.
- Point-to-point messaging uses a *message queue* to decouple applications.

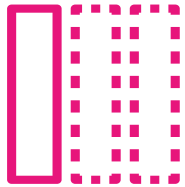


# Amazon Simple Queue Service (Amazon SQS)

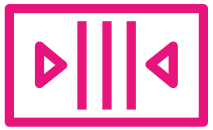
Amazon SQS

- Is a fully managed message queueing service
- Helps integrate and decouple distributed software systems and application components
- Provides highly available, secure, and durable message-queueing capabilities
- Provides an AWS Management Console interface and a web services API

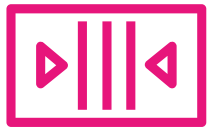
# Amazon SQS basic components



Message



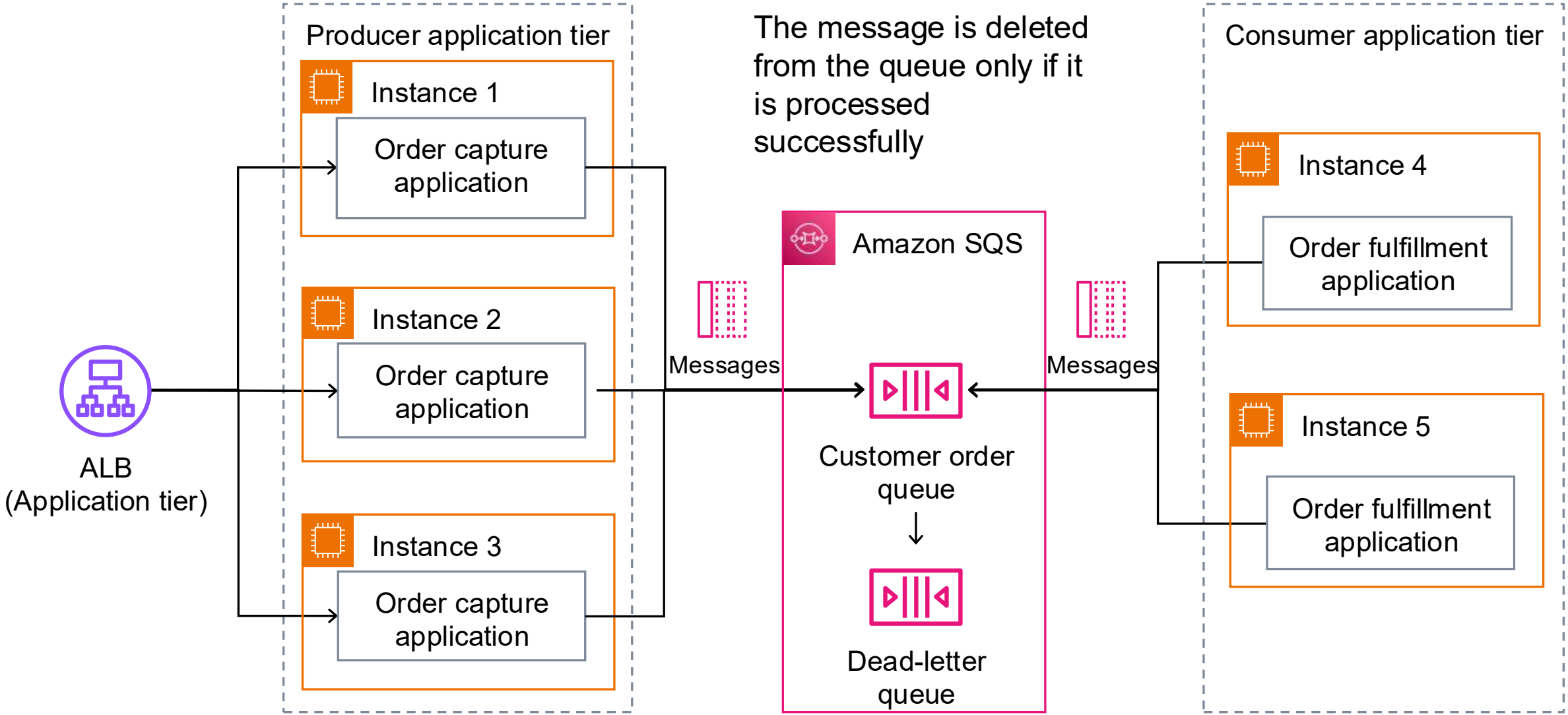
Queue



Dead-letter queue

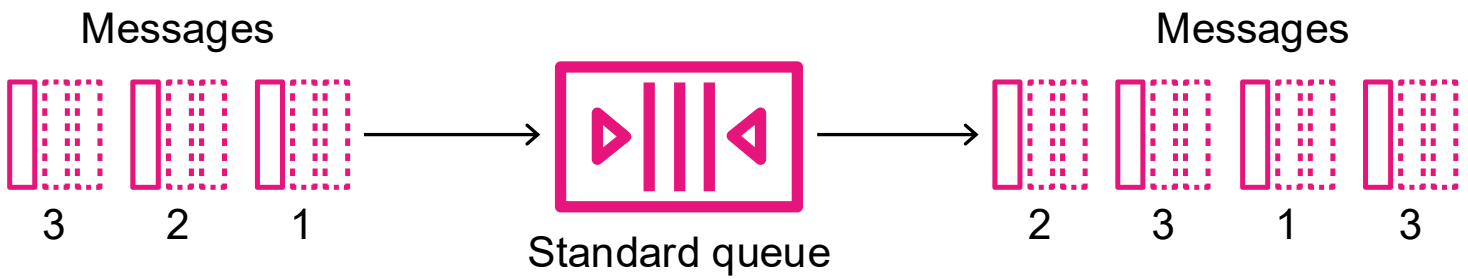
- A message can be up to 256 KB in size.
- A message remains in a queue until it is explicitly deleted or exceeds the queue's message retention period.
- Amazon SQS offers two types of queues: standard and first-in-first-out (FIFO)
- You can configure queue parameters, including the following:
  - Message retention period
  - Visibility timeout
  - Receive message wait time (short polling versus long polling)
- You can associate a dead-letter queue (DLQ) with any queue.
- A DLQ stores messages that cannot be consumed successfully.

# Decoupling example: Amazon SQS



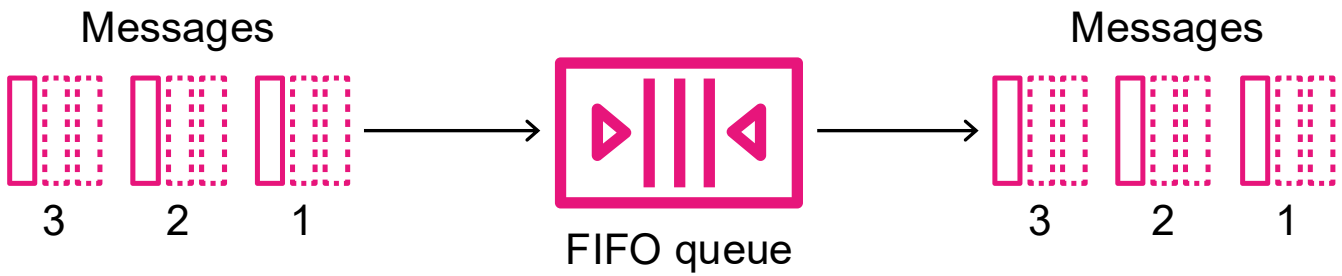
# Queue types

## Standard queue




- At-least-once delivery
- Best-effort ordering
- Nearly unlimited throughput

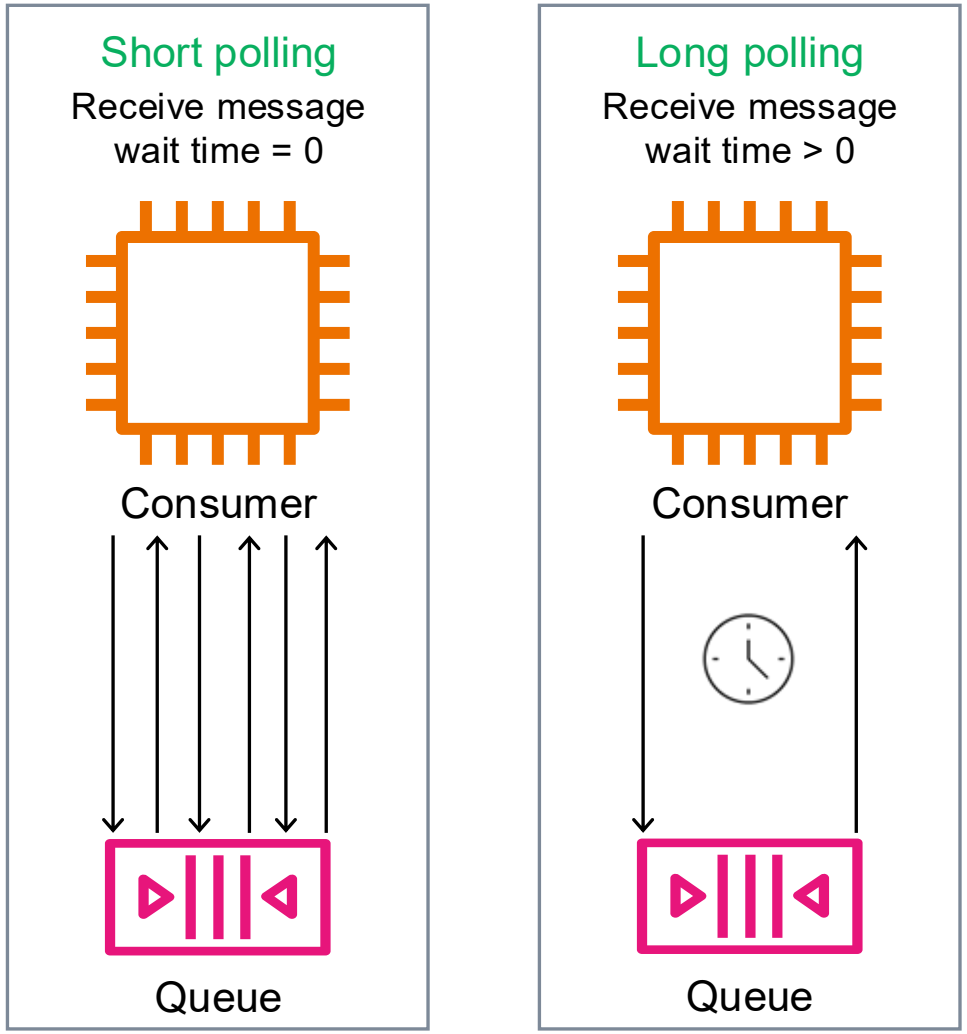
## FIFO queue




- FIFO delivery
- Exactly once processing
- High throughput

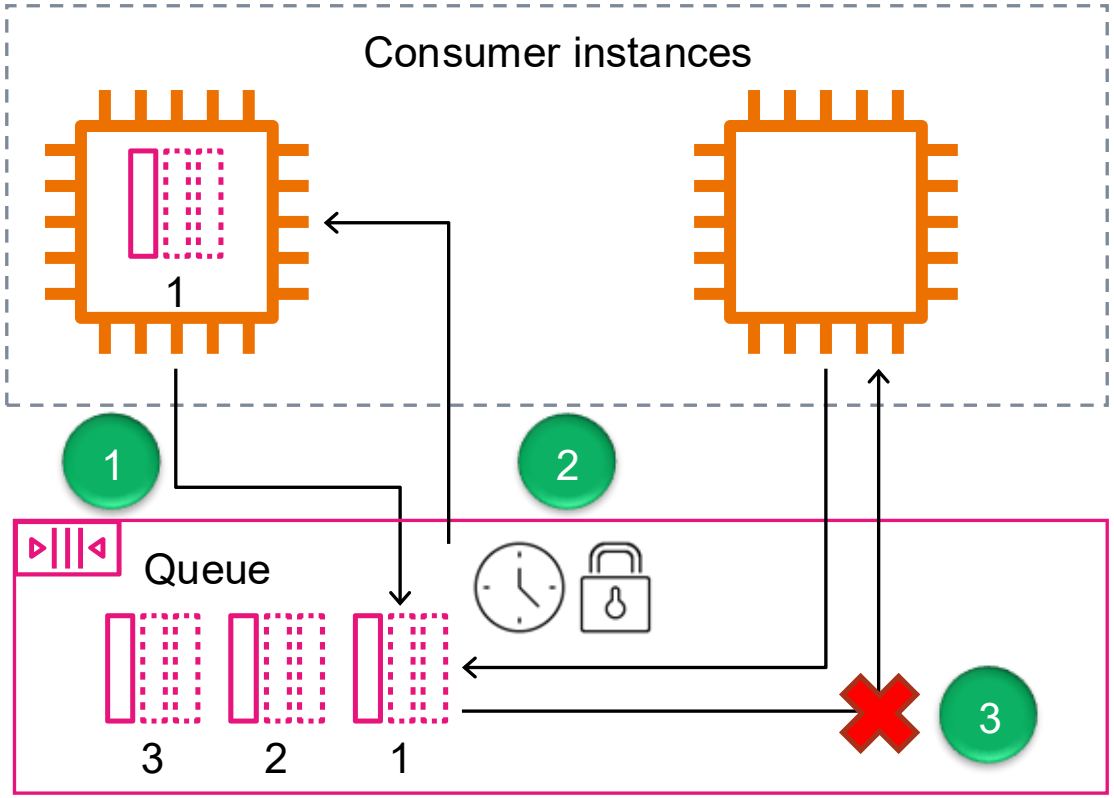
# Queue configuration: Polling type

Choose the right polling type.

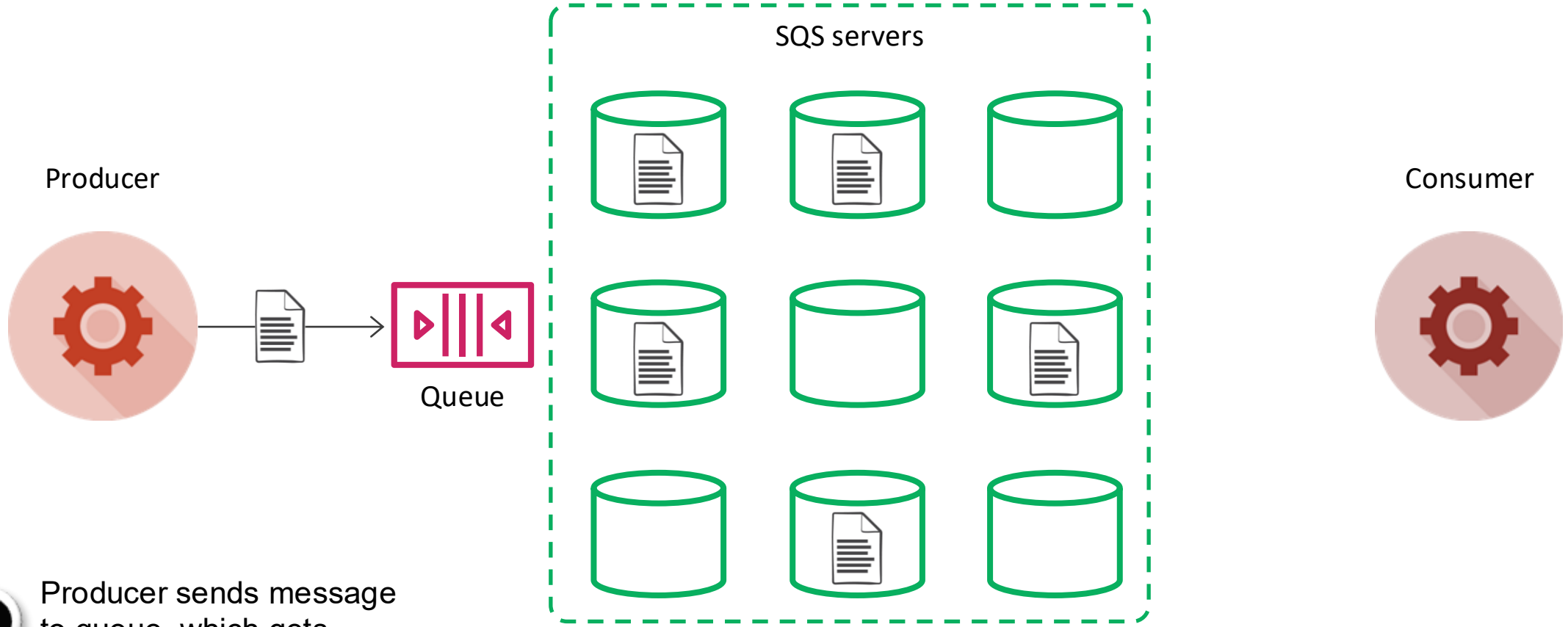


# Queue configuration: Message visibility

Tune your visibility timeout.

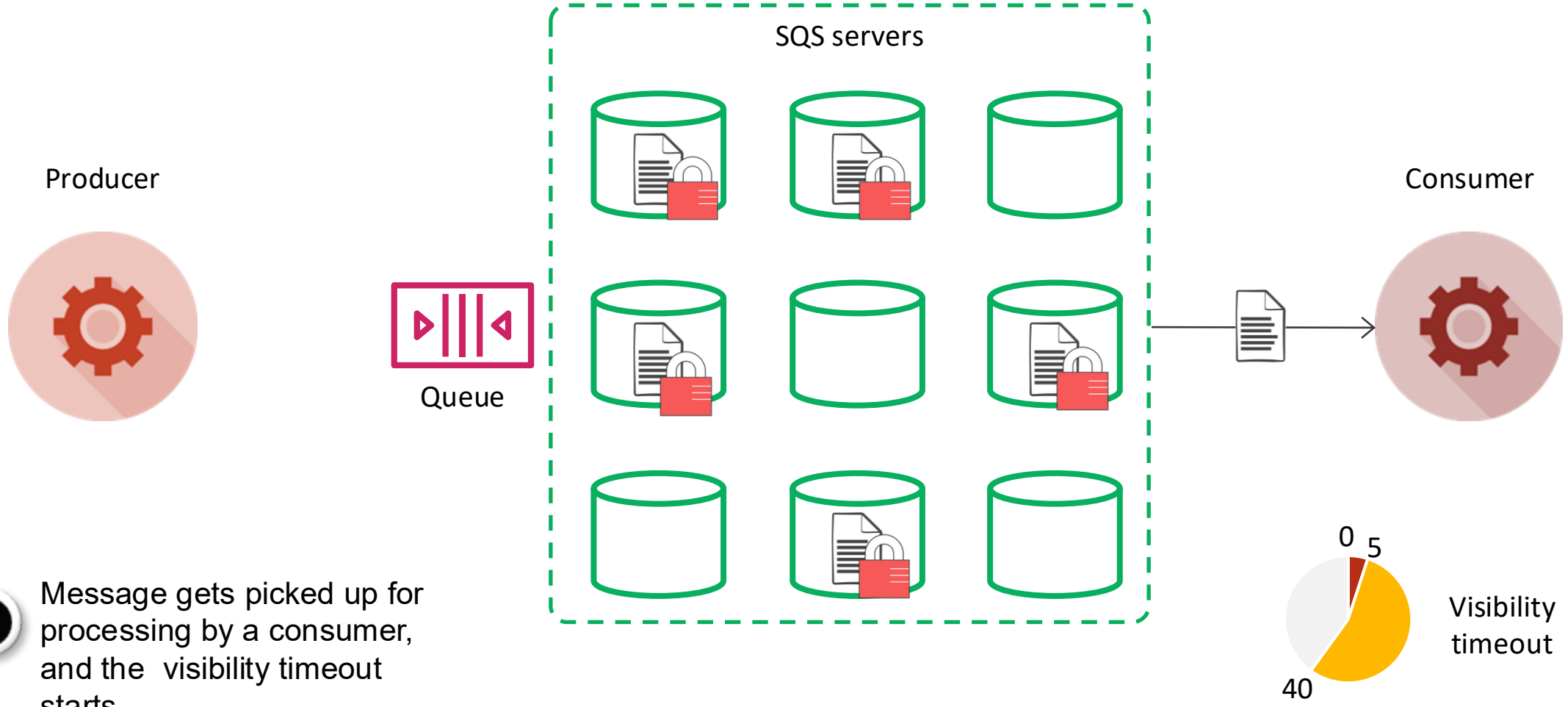


# Amazon SQS message lifecycle: Create



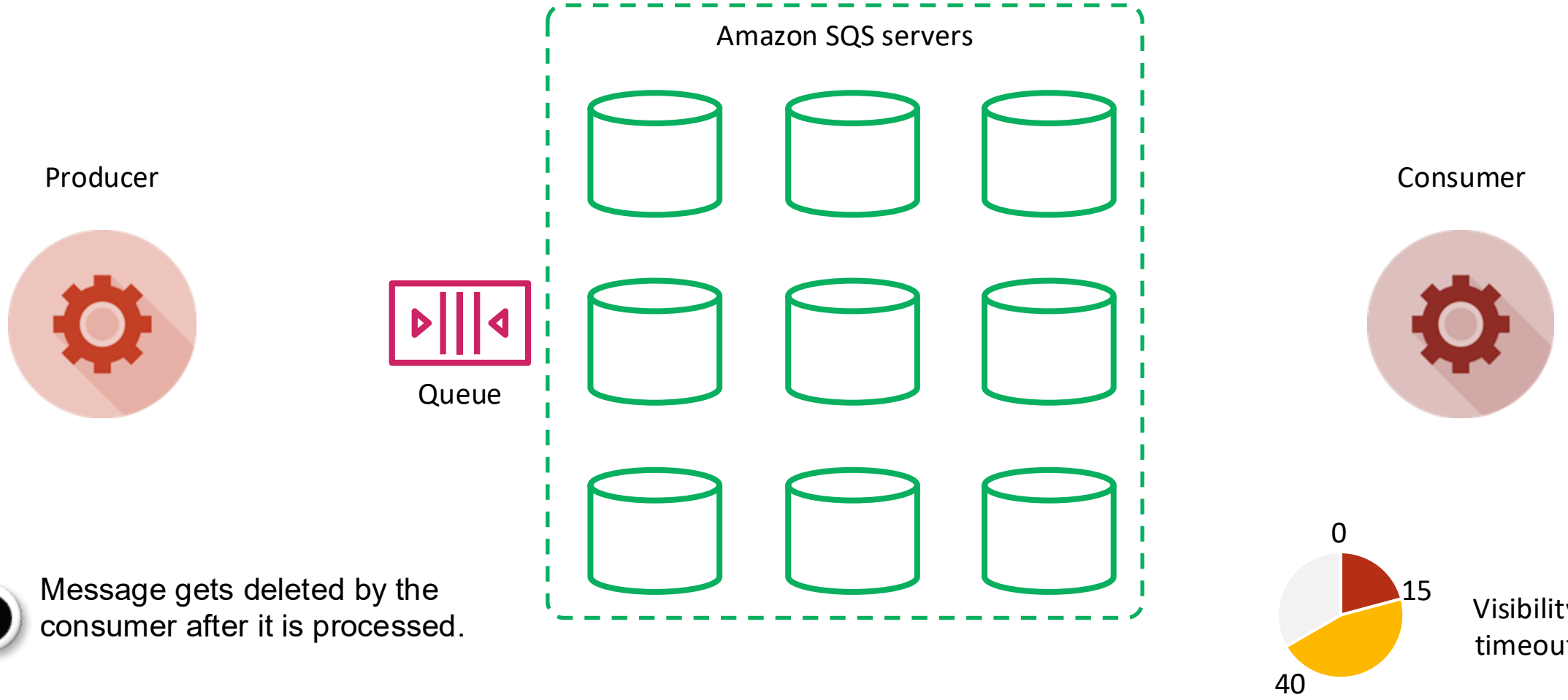

1 Producer sends message to queue, which gets distributed redundantly.

# Amazon SQS message lifecycle: Process




2 Message gets picked up for processing by a consumer, and the visibility timeout starts.

# Amazon SQS message lifecycle: Delete




3 Message gets deleted by the consumer after it is processed.

# SQS sample scenario

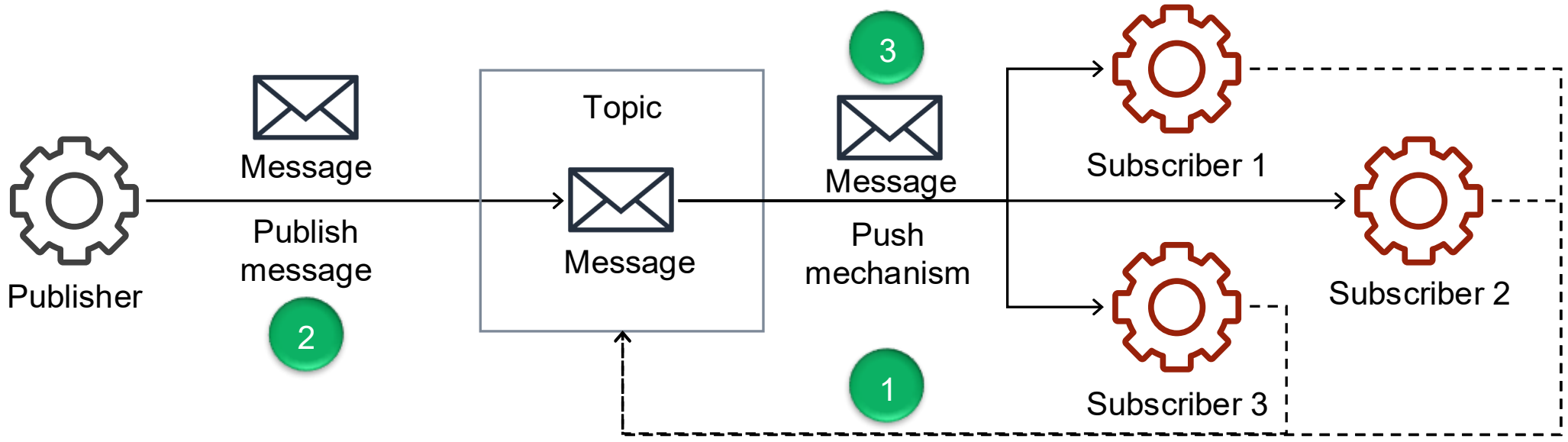
- Consider an Amazon SQS standard queue with three messages:
- The queue has a visibility timeout of 10 seconds.
- There are two consumers, **C1** and **C2**, both polling the queue.
- At time t = 0, C1 calls `ReceiveMessage` and receives **M1**. C1 starts processing **M1**, but processing will take 15 seconds.
- At time t = 2, C2 calls `ReceiveMessage`.
  - Can C2 receive M1 at t = 2? Why or why not?
  - If the queue uses short polling, is C2 guaranteed to receive M2?
  - If the queue uses long polling, is C2 guaranteed to receive M2?
  - At time t = 10, C1 has not deleted M1 yet. What happens to M1?
  - At time t = 12, C2 calls ReceiveMessage again. Could C2 receive M1 even though C1 is still processing it?
  - Suppose C1 finishes at t = 15 and tries to delete M1. What issue might occur?

| Message | Arrival order |
|---|---|
| M1 | first |
| M2 | second |
| M3 | third |

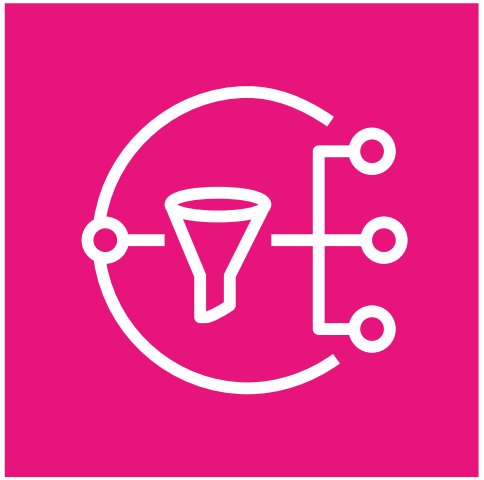
# Amazon SNS

# Publish/subscribe messaging

- You can decouple applications asynchronously by using publish/subscribe (pub/sub) messaging.
- Use pub/sub messaging when the sending application sends a message to multiple receiving applications and has little or no knowledge about the receiving applications.
- The sending application is called a *publisher*.
- The receiving application is called a *subscriber*.
- Pub/sub messaging uses a *topic* to decouple applications.



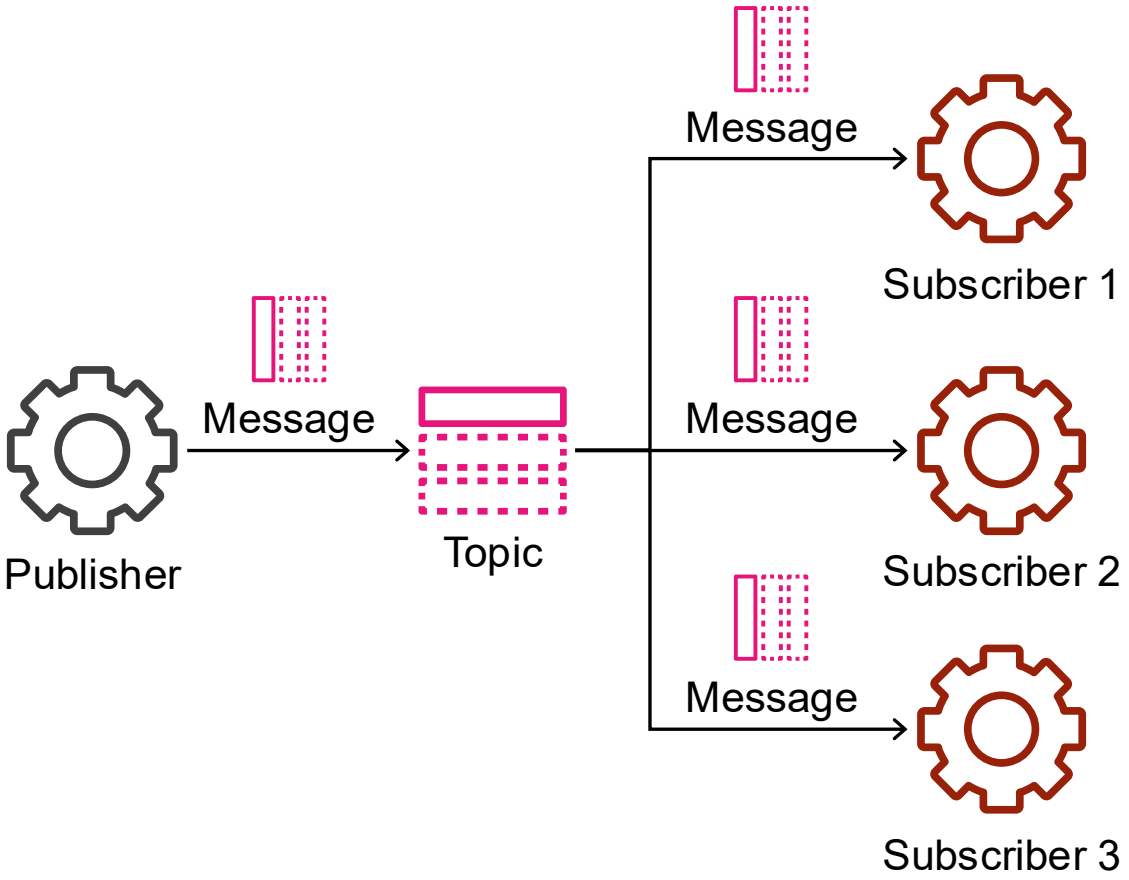
# Amazon Simple Notification Service (Amazon SNS)



Amazon SNS

- Is a fully managed pub/sub messaging service
- Helps decouple applications through notifications
- Provides highly scalable, secure, and cost-effective notification capabilities
- Provides an AWS Management Console interface and a web services API

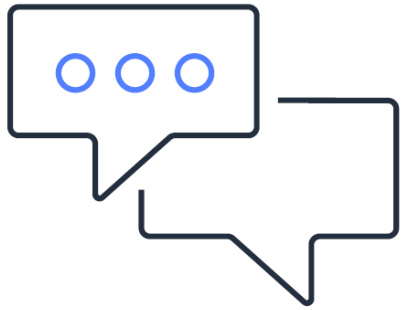
# Types of subscribers



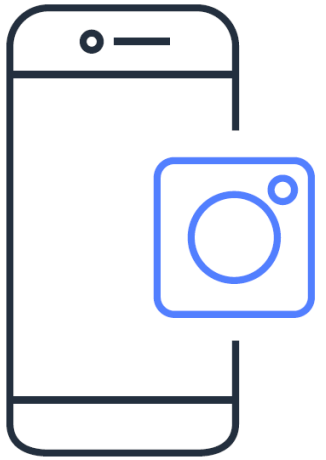

- Email destination
- Mobile text messaging destination
- Mobile push notifications endpoint
- HTTP or HTTPS endpoint
- AWS Lambda function
- SQS queue
- Amazon Kinesis Data Firehose delivery stream

# Amazon SNS use cases

Application and system alert notification



Email and text message notification



Mobile push notification

# Amazon SNS considerations

## Message publishing

- Single published message
- No recall options

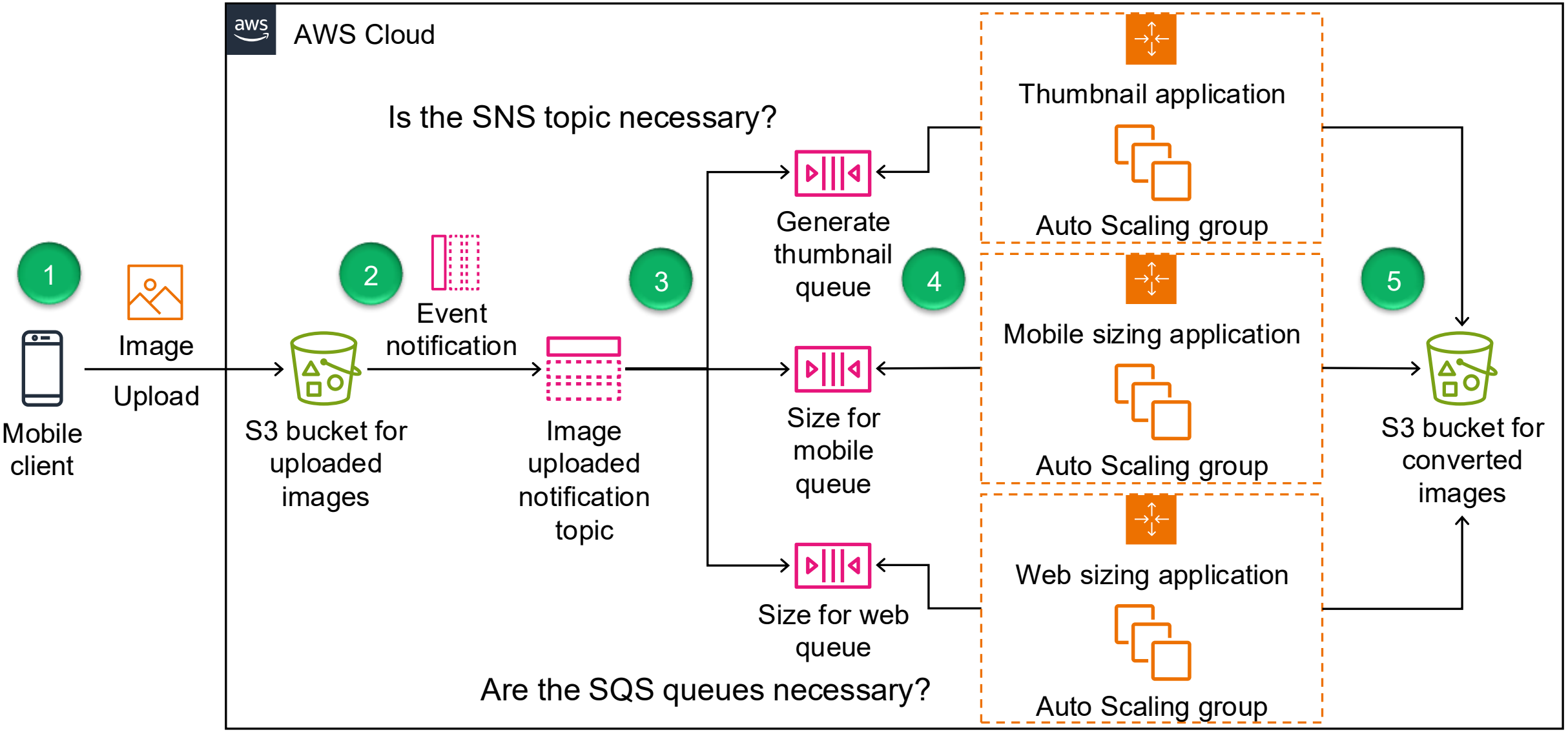
## Message delivery

- Use a *standard* topic if the message delivery order does not matter.
- Use a *FIFO* topic if an exact message delivery order is required.
- Customize the delivery policy of an HTTP or HTTPS endpoint to control the retry behavior.

# Amazon SNS and Amazon SQS comparison

| | Amazon SNS | Amazon SQS |
|---|---|---|
| **Messaging Model** | Publisher-Subscriber | Producer-Consumer |
| **Distribution Model** | One to many | One to one |
| **Delivery Mechanism** | Push (passive) | Pull (active) |
| **Message Persistence** | No | Yes |

# Decoupling example: Using Amazon SQS with Amazon SNS



# Permissions needed

- Action
- Who (principal)
- Where to configure (IAM role vs resource policy)

# 1. Client -> S3 (uploading image)

- Required permission (action)
  - `s3:PutObject` on upload bucket
- Who is calling
  - Either
    - Authenticated app user (via Cognito / backend) or
    - Backend service (API->S3)
- Where to set:
  - IAM policy (attached to role/user) and/or
  - S3 side: bucket policy

# 2. S3->SNS (event notification)

- Required permission
  - `sns:Publish` on the SNS topic
- Who is calling?
  - S3 service
- Where to set?
  - SNS topic resource policy

```
{
  "Effect": "Allow",
  "Principal": { "Service": "s3.amazonaws.com" },
  "Action": "sns:Publish",
  "Resource": "arn:aws:sns:...:image-upload-topic",
  "Condition": {
    "ArnLike": {
      "aws:SourceArn": "arn:aws:s3:::your-bucket-name"
    }
  }
}
```

# 3. SNS -> SQS (fan out to queue)

- Required permission
  - sqs:SendMessage
- Who is calling?
  - SNS service
- Where to set?
  - SQS queue resource policy

```
{
  "Effect": "Allow",
  "Principal": { "Service": "sns.amazonaws.com" },
  "Action": "sqs:SendMessage",
  "Resource": "arn:aws:sqs:...:queue-name",
  "Condition": {
    "ArnEquals": {
      "aws:SourceArn": "arn:aws:sns:...:topic-name"
    }
  }
}
```

# 4. EC2 -> SQS (Poll message)

- Required permission
  - sqs:ReceiveMessage
  - sqs:DeleteMessage
  - sqs:GetQueueAttributes
- Who is calling?
  - EC2 instances (thumbnail / resizing apps)
- Where to set?
  - IAM role (instance profile attached to EC2

# 5. EC2 -> S3 (get original image/ put processed one)

- Required permission
  - s3:GetObject
  - S3:PutObject
- Who is calling?
  - EC2 instances (thumbnail / resizing apps)
- Where to set?
  - Same IAM role (instance profile attached to EC2